Unidad 6. OpenLayers



Contenido de la unidad

- ✓ ¿Qué es OpenLayers?
- ✓ OpenLayers como librería JavaScript orientada a objetos.
- ✓ El mapa y la vista.
- ✓ Gestión de capas.
  - ✓ Capas de teselas.
  - ✓ Capas de imágenes.
  - ✓ Capas vectoriales.
- ✓ Controles.
- ✓ Interacciones.



mappingGIS

¿Qué es OpenLayers?



¿Qué es OpenLayers?

OpenLayers es una librería GIS *open source* desarrollada en JavaScript para construir **aplicaciones de mapas web** dinámicas.

Las aplicaciones de mapas contienen capas (ráster o vectoriales, que pueden proceder de una gran variedad de fuentes), y los controles para operar sobre esas capas.



mappingGIS

Un poco de historia

Creado en 2005 por Metacarta. **OpenLayers es un proyecto de OSGeo** (*Open Source Geospatial Foundation*).

Surgió como alternativa a Google Maps y otras APIs propietarias.

Año 2007: OpenStreetMap adoptó OpenLayers como visor de su cartografía.

Año 2013: OpenLayers 3: Reescritura completa en JavaScript moderno.

La popularidad de OpenLayers ha ido creciendo en los últimos años, mejorando la adición de nuevas características y por su estructura de módulos.

OpenLayers es una de las soluciones GIS más completas y potentes de código abierto para desarrollo web.

mappingGIS

Versiones

OpenLayers 2 (hasta 2.13) data del año 2005.

OpenLayers 3 surgió 2013 y supuso un salto cualitativo importante.

En junio de 2018 se liberó la versión 5.0 con mejora del soporte para imágenes vectoriales, tiles vectoriales y capas de imágenes dinámicas.

En septiembre de 2019 se liberó la versión 6.0, sin novedades significativas respecto a la versión 5.

En el año 2021 se lanza la versión 7: Mejor rendimiento, capas vectoriales con WebGL, enfoque en tiles vectoriales.

En 2023 aparece la versión 8 que supone una simplificación de la API , modularidad con ES Modules y mejor integración con frameworks como React/Vue.

En 2024- Lanzamiento de la versión 10, con rendimiento mejorado y una experiencia de desarrollo más fluida.

OpenLayers es compatible con una gran variedad de tipos de datos y capas. En OpenLayers se utilizan las últimas tecnologías de navegación como **HTML5**, **WebGL** y **CSS3**.

Para verlo necesitamos cualquier navegador web.



mappingGIS

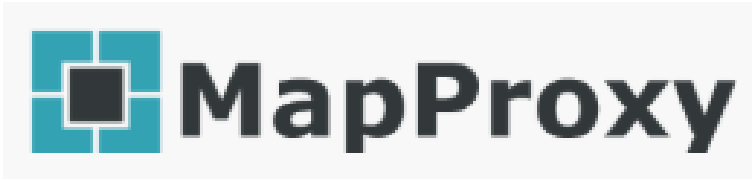
¿Qué podemos hacer con OpenLayers?

Implementa estándares de la OGC. A través de servidores espaciales:

- ✓ MapServer
- ✓ Geoserver
- ✓ MapProxy
- ✓ ArcGIS Server







ArcGIS Server

¿Qué podemos hacer con OpenLayers?

- Leer y escribir muchos **formatos** de datos:
  - WMS, WFS, GML, KML, GeoJSON, TopoJSON, teselas vectoriales de Mapbox....



- Trabajar con diferentes **proyecciones**
- Trabajar con **capas teseladas** de OSM, Bing, MapBox, Stamen, CARTO...
- Dar **simbología.**
- Incluir **controles.**
- Trabajar con **eventos**, etc...
- Customizar y extender con plugins.





mappingGIS

Documentación

Documentación:
https://openlayers.org/doc/quickstart.html

Documentación de la API:
http://openlayers.org/en/latest/apidoc/

Ejemplos (+ 260):
http://openlayers.org/en/latest/examples/

Código:
https://github.com/openlayers/openlayers

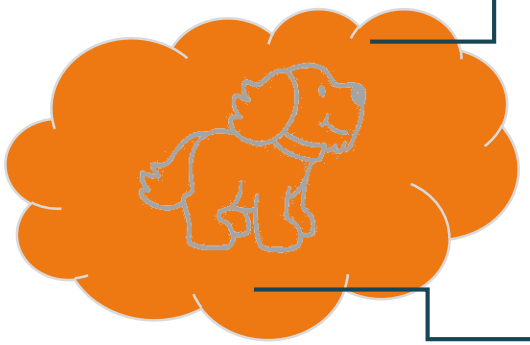
mappingGIS

OpenLayers como librería JavaScript orientada a objetos



OpenLayers como librería JavaScript orientada a objetos

CLASE PERRO
(abstracto)



PROPIEDADES
- Raza
- Nombre
- Peso
- Color
- Etc.

MÉTODOS
- Correr
- Saltar
- Comer
- Morder
- Etc.

OBJETO PERRO
(Clase instanciada)



PROPIEDADES
- Mastín Asiático
- Ludo
- 15 Kg
- Marrón canela
- Etc.

MÉTODOS
- Corre
- Salta
- Come
- Muerde
- Etc.

mappingGIS

OpenLayers como librería JavaScript orientada a objetos

CLASE ol.View (abstracto)



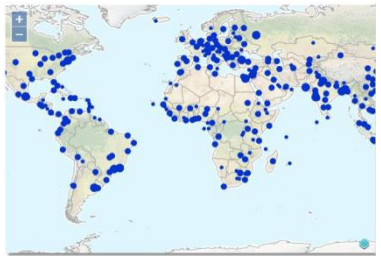
PROPIEDADES

Center
Zoom
Resolution
Rotation

MÉTODOS

Fijar un zoom
Obtener ubicación

OBJETO 'vista' (Clase instanciada)



PROPIEDADES

Zoom 6
Proyección EPSG:4326
Centro de ubicación 41.5, -4.65

MÉTODOS

```
vista.setZoom (4);
vista.setCenter ([2.18, 41.38]);
vista.setResolution (12000);
```

mappingGIS

OpenLayers como librería JavaScript orientada a objetos

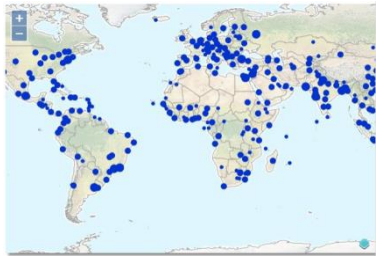
CLASE ol.View (abstracto)



```
var vista = new ol.View ({
    center: [0, 0],
    zoom: 2
});

vista.setZoom (6);
vista.setCenter ([-4.72, 41.66]);
```

OBJETO vista (Clase instanciada)



Métodos



https://openlayers.org/en/latest/apidoc/module-ol_View-View.html

mappingGIS

Comenzando con OpenLayers – Carga de la librería

El encabezado del documento carga necesariamente los archivos JavaScript y CSS que serán incluidos con el mapa:

- ✓ La librería JavaScript de OpenLayers (**ol.js**).
- ✓ Seguido por una hoja de estilo por defecto (**ol.css**) que especifica el estilo.

```
<!doctype html>
<html lang="en">
  <head>
    <script src="https://cdn.jsdelivr.net/npm/ol@v10.5.0/dist/ol.js"></script>
     <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/ol@v10.5.0/ol.css">

    <style>
      #map {
        height: 400px;
        width: 100%;
      }
    </style>
  </head>
...
```

Comenzando con OpenLayers – Carga de la librería

Dos maneras de utilizar la API de OpenLayers:

1. Utilizando la librería online, guardada en el servidor de OpenLayers **(no recomendado para producción)** https://openlayers.org/en/latest/build/ol.js

2. Descargando la librería:

   A. La distribución completa se puede descargar desde https://github.com/openlayers/openlayers

   B. Instalando el paquete **ol**. Para crear la aplicación necesitamos tener instalado node.js.

```
npm install ol
```

mappingGIS

El mapa y la vista



El mapa y la vista

El mapa ≠ **La vista**

El **mapa** tiene el control sobre las capas, controles, superposiciones, etc

Un mapa está hecho de:

- ✓ **Capas**
- ✓ **Una vista**
- ✓ **Controles**
- ✓ **Interacciones**

La **vista** es la forma en la que visualizamos el mapa.

La vista es como una ventana a través de la que vemos el mapa. Permite cambiar la ubicación que estamos buscando o de lo que está más cerca o más lejos.

mappingGIS

El mapa – ol.Map

ol.Map es la **clase que permite manejar el concepto de mapa** en nuestra aplicación. Podemos añadir o eliminar capas, controles, superposiciones e interacciones (*layers, controls, overlays* e *interactions*).

En su forma más simple, la creación de una nueva instancia ol.Map requiere que el usuario especifique **un objeto** con las siguientes **propiedades**:

- **target** el elemento HTML de destino, donde se representará el mapa.
- **layers** una o más referencias de la capa con los datos que se mostrarán.
- **view** una instancia ol.View es responsable de gestionar la forma de visualizar el mapa.

```
var map = new ol.Map({
    target: …
    layers: …
    view:
});
```

El mapa – ol.Map

Para crear un mapa, primero debemos **crear el espacio que ocupara en nuestra web**. Lo más sencillo es crear un bloque `<div>`. De este modo generamos un único elemento de documento:

```
<div id="map"></div>
```

Este elemento servirá como contenedor para la vista del mapa.

También damos al contenedor un atributo id para que podamos hacer referencia a él en nuestro código.

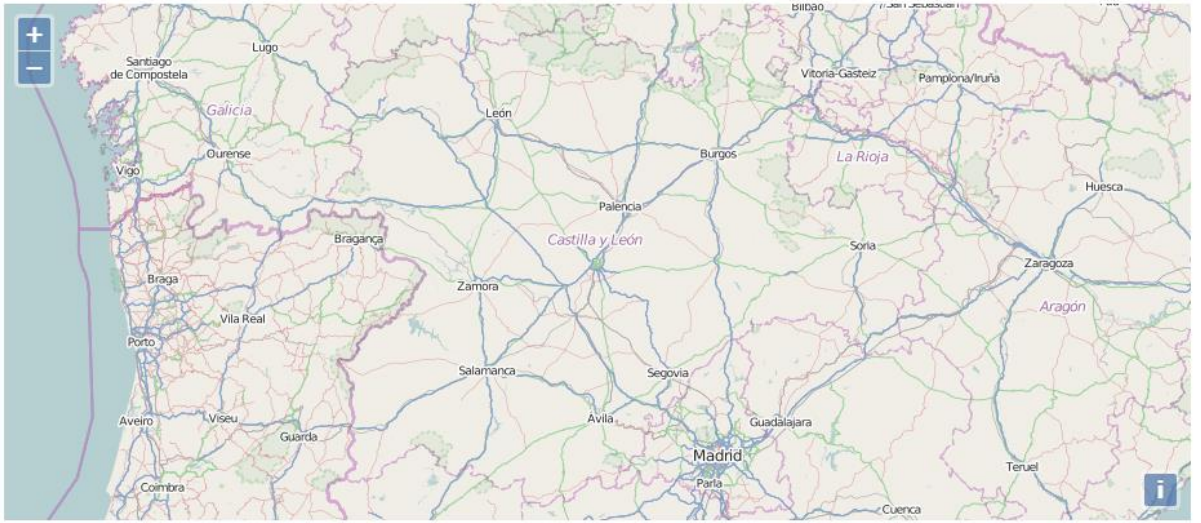
```
var map = new ol.Map();
```

Este **constructor** (referido al nombre de la variable JavaScript `map`) va a crear el mapa en el elemento `<div>` creado anteriormente.

El mapa – ol.Map

Un mapa básico puede crearse fácilmente con el siguiente código:

```
var map = new ol.Map({
    target: 'map',
    layers: [
        new ol.layer.Tile({source: new ol.source.OSM()})
    ],
    view: new ol.View({
        center: ol.proj.fromLonLat([-4.72, 41.66]),
                zoom: 7
    })
});
```



mappingGIS

Propiedades de la clase ol.Map

4 propiedades fundamentales:

- target
- layerGroup
- view
- size

Otras opciones secundarias:

- controls
- interactions
- logo
- overlays
- renderer

mappingGIS

Métodos de la clase ol.Map

La clase ol.Map ofrece **métodos** para obtener y establecer sus **propiedades**, además de los métodos necesarios para trabajar con capas, controles, etc.

Todas las propiedades del mapa tienen sus correspondientes métodos get y set:

- `getTarget, setTarget,`
- `getView, setView,`
- `getSize, setSize,`
- `getLayerGroup y setLayerGroup`

```
var size = map.getSize();
var view = map.getView();
var layers = map.getLayers();
```

mappingGIS

Renderizando el mapa

DOM **Canvas** WebGL

- Usando el renderizado **DOM**, ***Document Object Model*** todos los elementos del mapa se dibujan usando **elementos HTML**, por ejemplo usando img para las teselas o svg para objetos geográficos.

- Con **canvas**, los mapas se representan mediante **el elemento canvas de HTML5**, que ofrece una forma de secuencias de comandos para hacer formas 2D e imágenes de mapa de bits.

- **WebGL** renderiza el mapa utilizando la tecnología WebGL, un subconjunto del estándar OpenGL adecuado para los navegadores, y que permite aprovechar el potencial de la tarjeta gráfica (la GPU).

```
var map = new ol.Map ({
            target:…
    renderer: 'canvas',
    layers:..
    view…
});
```

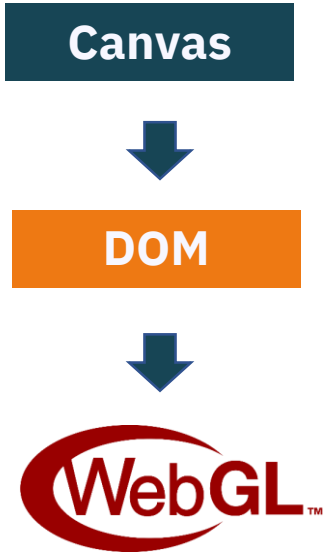
Podemos forzar al mapa a utilizar un renderizador específico utilizando las **propiedad renderer**.

mappingGIS

Renderizando el mapa

Secuencia de renderizado en OpenLayers por defecto:

```
['canvas' , 'dom' , 'webgl' ]
```



Esto significa que cuando se crea una nueva instancia de mapa, OpenLayers 3 intenta utilizar el procesador **canvas** si el navegador lo soporta, en caso contrario intenta utilizar la tecnología **DOM** y, por último, si tampoco es soportado utiliza el mecanismo **WebGL**.

El mecanismo DOM tiene el peor rendimiento.

mappingGIS

La vista- ol.View

`ol.Map` no es responsable de cosas como fijar centro, el nivel de zoom, la proyección del mapa o el ángulo de rotación.

En su lugar, éstas son propiedades de un objeto **`ol.View`** – que representa una vista en 2D del mapa. La razón de esta abstracción es la idea de cambiar instantáneamente por ejemplo, entre un mapa en 2D y una vista inclinada en 3D, sin necesidad de mantener dos copias de las capas.

Cuando instanciamos una nueva vista necesitamos especificar una ubicación de centro (**center**) y nivel de **zoom** (o un valor de *resolution*) para tener una instancia de vista completa:

```
var view = new ol.View({
    center: ol.proj.fromLonLat([-4.72, 41.66]),
    zoom: 7
});
```

Propiedades y métodos de la clase ol.View

ol.View → Propiedades

- **center**
- **resolution**
- **rotation**

Cada propiedad tiene su correspondiente método con el get y el set.:

```
getCenter(), getProjection(), getResolution(), getRotation(), getZoom()
setCenter(center), setResolution(resolution), setRotation(rotation), setZoom(zoom)
```

Con una nueva instancia de view...

```
var view = new ol.View({
    projection: 'EPSG:4326',
    center: [2.1833, 41.3833],
    zoom: 5
```

... ya podemos utilizar un método para fijar el centro de la ubicación y un zoom:

```
view.setCenter([2.1833, 41.3833]);
view.setZoom(7);
```

mappingGIS

Otras propiedades de la vista –ol.View

Propiedad de proyección (`projection`) → Determina el sistema de coordenadas de center y las unidades para los cálculos de resolución del mapa

```
var view = new ol.View({
    projection: 'EPSG:4326'
});
```

Por defecto → Esférica de Mercator (EPSG:3857)

La **opción de zoom** es una manera conveniente para especificar la resolución del mapa.

Los niveles de zoom disponibles son determinados por:

- **maxZoom** (por defecto: 28),
- **zoomFactor** (por defecto: 2) y
- **maxResolution** (por defecto se calcula de tal manera que la validez de la medida se ajusta a la proyección en una tesela de 256x256 pixel).

Comenzando desde el nivel de zoom 0 con una resolución de maxResolution unidades por píxel, los niveles de zoom posteriores se calculan dividiendo la resolución del nivel de zoom anterior zoomFactor, hasta que se alcance el nivel de zoom maxZoom.

mappingGIS

Gestión de capas

mappingGIS

Las capas -Layers

Las ***Layers*** son contenedores ligeros que obtienen sus datos a partir de fuentes *(sources)*.

Una capa es una representación visual de datos de una fuente.

`ol.layer.Tile`

Fuentes de capas que proporcionan **imágenes teseladas** pre-renderizados en mallas que se organizan por niveles de zoom para resoluciones específicas.

`ol.layer.Image`

Para **imágenes renderizadas** de servidores que están disponibles para resolucion y extensiones arbitrarias.

`ol.layer.Vector`

Datos vectoriales que se renderizan en el lado del cliente

mappingGIS

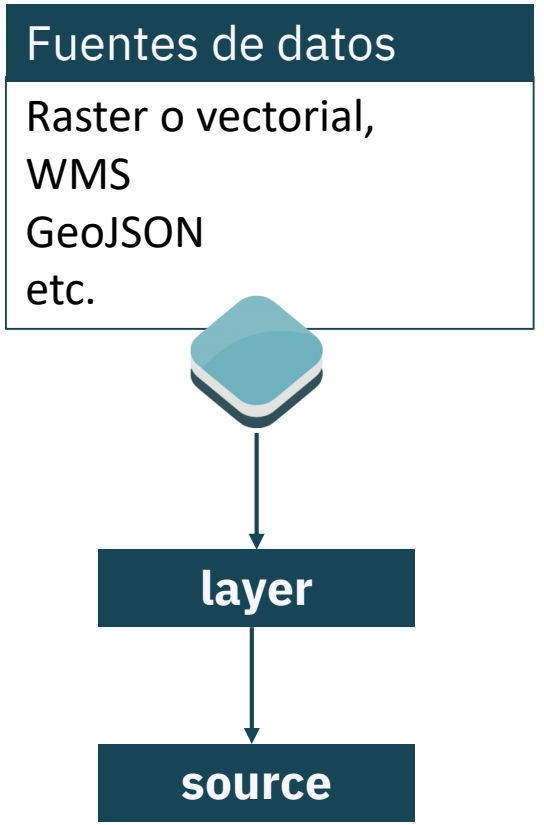
Las capas -Layers

OpenLayers diferencia el concepto de:

- **layer**: representación de un conjunto de datos.
- **source**: responsable de obtener los datos que serán usados por la capa.

source es el protocolo utilizado para obtener los mapas teselados.

- ❑ Tile sources para capas teseladas
- ❑ Image sources para capas raster
- ❑ Vector sources para capas vectoriales
- ❑ Vector tile sources para teselas vectoriales




```
var osmLayer = new ol.layer.Tile({
        source: new ol.source.OSM()
});
```

mappingGIS

Gestión de capas -Layers

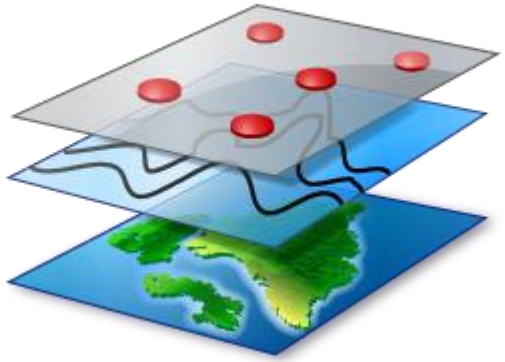
El mapa es un contenedor donde podemos apilar capas.

Las capas en **posición más alta tapan a las capas en posición más baja.**

Cada vez que añadimos una capa al mapa se añade a lo alto.

Podemos establecer las capas del mapa en el momento en el que instanciamos el mapa, para ello usamos la propiedad **layers** que acepta una lista de capas:

```
var map = new ol.Map({
    ...
    layers: [], // Instancia de mapa sin capas
    ...
});
```



Apilamiento de capas

mappingGIS

Gestión de capas -Layers

Métodos para **agregar** y **eliminar** capas de ol.Map

- `addLayer`
- `removeLayer`

```
var cities = ... // Una capa instanciada
var countries = ... // Otra capa instanciada

// Añadiendo capas al mapa
map.addLayer(cities);
map.addLayer(countries);


// Eliminamos una capa del mapa
map.removeLayer(countries);
```





mappingGIS

Propiedades de las capas



Herencia de la clase base `ol.layer.Base`.

La lista de las **propiedades más comunes** son:

- **`visible`**, un valor booleano que especifica si la capa debe dibujarse o no. Por defecto es true.
- **`opacity`**, un valor entre 0,0 y 1,0 que determina la opacidad de la capa. Por defecto 1.
- **`maxResolution`**, la resolución máxima a la que la capa debe dibujarse.
- **`minResolution`**, la resolución mínima a la que la capa debe ser dibujada.

Capas teseladas – ol.layer.Tile

clase ol.layer.Tile → Para las fuentes de capas que proporcionan teselas de imágenes pre-renderizadas en mallas que se organizan por niveles de zoom para resoluciones específicas.

Para crear una nueva instancia necesitamos especificar la propiedad **source** que usaremos:

```
// Carga de capas teseladas de un servidor WMS
     var  wmsLayer = new ol.layer.Tile({
               source: new ol.source.TileWMS({
               url: 'http://maps.opengeo.org/geowebcache/service/wms',
               params: {LAYERS: 'bluemarble',
               VERSION: '1.1.1',
             STYLES: 'default'}
               })
     })
//Carga de Capas teseladas del servidor de Bing
var BingLayer = new ol.layer.Tile({
     source: new ol.source.BingMaps({
          key: ‘xx',
          imagerySet:'Road'})
     })
];
```



mappingGIS

Fuentes de Capas teseladas – ol.layer.Tile

clase ol.layer.Tile

Fuentes

- `ol.source.TileWMS`
- `ol.source.ImageWMS`

Tanto `ol.source.ImageWMS` y `ol.source.TileWMS` aceptan los siguientes parámetros:

- **`attributions, crossOrigin, logo, ratio, resolutions`**
- **`url`**: La URL del servicio WMS que será consultado
- **`params`**: es un objeto JavaScript donde podemos especificar muchos de los parámetros de la operación ***GetMap***, como son **LAYERS** (obligatorio), **VERSION** (por defecto 1.3.0), **TRANSPARENT**, **STYLES, WIDTH, HEIGHT, BBOX, CRS**.

Otras fuentes para proveedores de teselas:

- `ol.source.OSM`
- `ol.source.MapQuest`
- `ol.source.Stamen`
- `ol.source.BingMaps`
- `ol.source.XYZ`



mappingGIS

Capas de imágenes – ol.layer.Image

clase ol.layer.Image + ol.source.ImageStatic

```
var ImageLayer = new ol.layer.Image({
    source: new ol.source.ImageStatic({
            url: 'http://imgs.xkcd.com/comics/online_communities.png',
    })
})
```

Las capas de imagen son apropiadas para trabajar con imágenes ligeras. Para grandes imágenes es mejor utilizar un mapa del servidor para servir teselas calculadas a partir de la gran archivo original.

```
var ImageLayer = new ol.layer.Image({
    source: new ol.source.ImageWMS({
        url: 'http://localhost:8080/geoserver/wms',
        params: { 'LAYERS': 'earthgroup', 'FORMAT': 'image/jpeg' },
    })
})
```

mappingGIS

Capas vectoriales – ol.layer.Vector

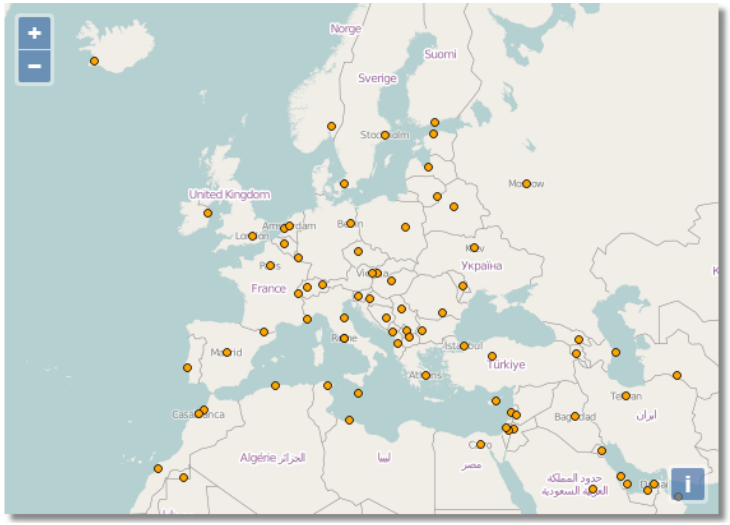
La clase **ol.layer.Vector** **nos permite trabajar con datos vectoriales**.

Las capas vectoriales son complejas. Las capas vectoriales podrían requerir crear nuevos elementos, editar o borrar los existentes y enviar los cambios al servidor para que se mantengan.

En la forma más básica, podemos crear una nueva instancia **ol.layer.Vector** simplemente especificando la propiedad **source** responsable de cargar los datos.

Ejemplo:

```
var vector = new ol.layer.Vector({
  source: new ol.source.Vector({
     url: ... //ubicación del archivo GeoJSON
     format: new ol.format.GeoJSON()
   })
});
```



mappingGIS

Capas vectoriales – ol.layer.Vector

OpenLayers nos ofrece la clase fuente ol.source.Vector para leer información vectorial para los formatos de datos mas comunes:

- `ol.format.GeoJSON`
- `ol.format.TopoJSON`
- `ol.format.GPX`
- `ol.format.KML`
- `ol.format.OSMXML`

mappingGIS

Capas teseladas vectoriales – ol.layer.VectorTile

clase **ol.layer.VectorTile** → Datos Vectoriales teselados= Los datos vectoriales se sirven en una malla teselada.

ol.layer.VectorTile + ol.source.VectorTile

Aunque esta fuente recibe teselas con objetos geográficos vectoriales del servidor, no es adecuado para edición de los objetos geográficos. Dichos objetos geográficos son optimizados para una **rápida renderización**: las geometrías se recortan en el límite del encuadre del mapa y se simplifican para una resolución de vista.

Un mapa de teselas vectoriales reutiliza las mismas teselas de origen para niveles de zoom próximos y así ahorrar ancho de banda en **dispositivos móviles.**

mappingGIS

Capas teseladas vectoriales – ol.layer.VectorTile

El formato utilizado para representar las teselas vectoriales es ***ol.format.MVT***

Ejemplo

```
var vectortile = new ol.layer.VectorTile({
    source: new ol.source.VectorTile({
        format: new ol.format.MVT(),
        url: 'https://{a-d}.tiles.mapbox.com/v4/mapbox.mapbox-streets-v6/' +
                '{z}/{x}/{y}.vector.pbf?access_token=' + key
        })
}
```



mappingGIS

Heatmap layer – ol.layer.Heatmap



Mapa de Calor

Representación visual de los datos en la que se muestra una gama de colores para mostrar los datos, permitiendo conocer en qué lugares la concentración de datos es mayor.



Clase **ol.layer.Heatmap**

Subclase de **ol.layer.Vector**, que nos permite el acceso y representación de datos vectoriales.

Necesita que especifiquemos una instancia válida en la propiedad **source**.

mappingGIS

Heatmap layer – ol.layer.Heatmap

Propiedades de la clase **ol.layer.Heatmap**

- **gradient**: una lista con el conjunto de colores en formato hexadecimal que va a determinar el mapa de calor. Los valores por defecto son ['# 00f', '# 0ff', '# 0F0', '# ff0', '# f00'].
- **weight**: el campo que se utilizará como valor de peso. No es obligatorio.
- **radius**: el radio de los círculos, por defecto 8.
- **blur**: el tamaño del efecto de desenfoque, por defecto 15 px.
- **shadow**: el tamaño del efecto de sombra para cada círculo, por defecto 250 px.

Ejemplo

```
var heatmap = new ol.layer.Heatmap({
    source: new ol.source.Vector({
        url: 'cities.geojson‘
        format: new ol.format.GeoJSON()
        }),
    radius: 5
});
```

mappingGIS

Estilo de objetos geográficos vectoriales



Conjunto de clases para representar los objetos geográficos

clase **ol.style.Style**

Propiedades

- Fill (`ol.syle.Fill`)
- Stroke (`ol.style.Stroke`)
- Text (`ol.style.Text`)
- Image (`ol.style.Image`)

Estilo de objetos geográficos vectoriales

Opciones de clases de estilo de OpenLayers

`ol.style.Fill`: tiene la propiedad **`color`**. Solo necesita esta propiedad para especificar como rellenar una forma.

`ol.style.Stroke` acepta:

- `color`
- `width`

Para representar los objetos geográficos como imágenes, OpenLayers define dos clases: **`ol.style.Icon`** y **`ol.style.Circle`** que son herencias de la clase **`ol.style.Image`**

`ol.style.Icon` → para representar cualquier imagen especifica pasando una url a la propiedad image.

`ol.style.Circle` → para representar circunferencias con buen rendimiento, por eso se usa para renderizar objetos puntuales.

Estilo de objetos geográficos vectoriales

Código utilizado **por defecto** para representar los objetos geográficos vectoriales:

```
var fill = new ol.style.Fill({
 color: 'rgba(255,255,255,0.4)'
});
var stroke = new ol.style.Stroke({
     color: '#3399CC',
     width: 1.25
});
var styles = [
     new ol.style.Style({
          image: new ol.style.Circle({
               fill: fill,
               stroke: stroke,
               radius: 5
          }),
          fill: fill,
          stroke: stroke
     })
];
```

mappingGIS

Estilo de objetos geográficos vectoriales - Textos



El siguiente código muestra el atributo texto para representar el mensaje hola con un borde y un relleno:

```
text: new ol.style.Text({
    text: 'hola',
    fill: new ol.style.Fill({
        color: 'red'
    }),
    stroke: new ol.style.Stroke({
        color: 'white',
    width: 4
    )
})
```

mappingGIS

Funciones de estilo

Función de estilo → responsable de devolver un listado de estilos que serán aplicados a los objetos geográficos.

Cada vez que se representa este objeto se buscan los estilos disponibles.

Cuando una función de estilo se aplica a una capa **`ol.layer.Vector`** la función debe ser del tipo **`ol.style.StyleFunction`** que conforma la estructura:

```
function (ol.Feature, number): Array.<ol.style.Style>.
```

La siguiente es una función que sigue esta estructura:

```
var customStyleFunction = function(feature, resolution) {
  var style1 = ...;
  var style2 = ...;

return [style1, style2];
```

mappingGIS

Gestión de features

Las clases de origen vectorial ofrecen **métodos** para recuperar, agregar o eliminar objetos geográficos. Todos los métodos comunes que se requieren para administrar los *features* se implementan por la clase base **`ol.source.Vector`** :

- **`addFeature()`** permite añadir una función o un conjunto de elementos.
- **`clear()`** elimina todos los elementos.
- **`getFeatures()`** devuelve un listado con todos los elementos.
- **`getFeaturesAtCoordinate()`** devuelve una matriz de características que se cruza con el dado de coordenadas.
- **`removeFeature()`** elimina un elemento.
- **`getExtent(),`** devuelve la extensión ocupada por todos los elementos.
- **`forEachFeature()`** permite aplicar una función a cada elemento.

mappingGIS

Controles



Controles

En OpenLayers, un control es un *widget* visible con un elemento DOM en una posición fija en la pantalla.

Pueden implicar la acción del usuario mediante botones, o ser meramente informativos.

Controles por defecto:

- ✓ Atribución: `ol.control.Attribution`



- ✓ Logotipo: `ol.control.Rotate`
- ✓ Zoom: `ol.control.Zoom`



Controles

A menos que se configure de otro modo, de forma predeterminada se crea una instancia **`ol.control.defaults`** para cada uno de los siguientes controles:

- **Zoom**: **`ol.control.Zoom`** Para diseñar este control, utilizar los selectores css `.ol-zoom-in` y `.ol-zoom-out`.
- **Rotación**: **`ol.control.Rotate`**. Se trata de un botón de control para restablecer la rotación a 0. Para diseñar este control, usamos el selector css `.ol-rotate`. Se agrega un selector css `.ol-hidden` al botón cuando la rotación es 0.
- **Atribución**: **`ol.control.Attribution`**. Control para mostrar todas las atribuciones asociadas con las fuentes de las capas en el mapa. Por defecto, se mostrará en la parte inferior derecha del mapa, pero esto se puede cambiar usando un selector CSS para la atribución de `.ol-attribution`.

```
var map = new ol.Map({
    controls: ol.control.defaults({
         zoom: false
     }),
     ...
 });
```

Ejemplo para inicializar un mapa usando los controles por defecto pero deshabilitando el control de zoom:

mappingGIS

Controles

Resto de clases

- ol.control.FullScreen añade un botón que muestra el mapa a pantalla completa.
- ol.control.MousePosition muestra las coordenadas del ratón.
- ol.control.OverviewMap muestra en un pequeño mapa la ubicación en un zoom menor.
- ol.control.ScaleLine crea un control de escala.
- ol.control.ZoomSlider crea un zoom deslizante.
- ol.control.ZoomToExtent cambia la vista del mapa a una extensión especificada.

Podemos ampliar los controles de la siguiente forma:

```
var map = new ol.Map({
        controls: ol.control.defaults().extend( [
                        new ol.control.ZoomSlider(),
                        new ol.control.ScaleLine()
        ]),
        ...
});
```

mappingGIS

Interacciones



Interacciones

Son acciones del usuario que cambian el estado del mapa.

Al contrario que los controles, las interacciones NO tienen un elemento DOM asociado, por lo que no podemos modificar el estado del mapa apretando un botón, se aplican directamente utilizando el ratón o el teclado (o la pantalla táctil).

La clase `ol.interaction.Interaction` es la responsable de manejar la interacción del usuario, aunque es un clase abstracta.

```
var int_dragbox = [
    new ol.interaction.DragBox()
 ];

// Inicialización de la interacción cuando construimos el mapa
var map = ol.Map({
    interactions: int_dragbox,
        ...
});
```

Interacciones predefinidas

Interacciones incluidas **por defecto** → `ol.interaction.defaults()`

Pueden excluirse del mapa mediante el establecimiento de la correspondiente opción a **false.**

El conjunto de interacciones por defecto son:

- `ol.interaction.DoubleClickZoom`: Permite al usuario acercarse un nivel de zoom haciendo doble clic en el mapa.
- `ol.interaction.DragPan:` Nos permite mover el mapa arrastrándolo. Debemos mantener pulsado el botón derecho del ratón.
- **ol.interaction.DragZoom**. Permite al usuario ampliar el mapa en una zona específica, arrastrando el ratón y seleccionando una caja. Normalmente se combina con `ol.events.condition` que lo limita a tener que mantener una tecla pulsada (tecla *Shift* de forma predeterminada).

mappingGIS

Interacciones predefinidas

- `ol.interaction.DragRotate`: Permite rotar el mapa haciendo clic y arrastrando en el mapa. Sólo es compatible con dispositivos que tienen un ratón. Funciona presionando **shift + Alt + arrastre del ratón**

- `ol.interaction.KeyboardPan`. Permite explorar el mapa utilizando las flechas del teclado. Aunque esta interacción está incluida por defecto en los mapas, las teclas sólo pueden ser utilizadas cuando el foco del navegador está en el elemento al que están unidos los eventos de teclado.

- `ol.interaction.KeyboardZoom`. Permite ampliar el mapa con las teclas del teclado + y -. Aunque esta interacción está incluida por defecto en los mapas, las teclas sólo pueden ser utilizadas cuando el foco del navegador está en el elemento al que están unidos los eventos de teclado.

- `ol.interaction.MouseWheelZoom`. Permite acercarse o alejarse del mapa moviendo la ruleta del ratón.

- `ol.interaction.PinchRotate`: En pantallas táctiles permite girar el mapa retorciéndolo con dos dedos.

- `ol.interaction.PinchZoom`. En pantallas táctiles permite ampliar el mapa pellizcando con dos dedos.

mappingGIS

Otras interacciones

No están por defecto

- `ol.interaction.DragBox`. Permite dibujar una caja vectorial arrastrando en el mapa. En combinación con otras interacciones como `ol.interaction.Select` que se puede utilizar para seleccionar los elementos que intersecten con la caja.

- **ol.interaction.DragRotateAndZoom** Es una mezcla entre **`ol.interaction.DragZoom`** y **`ol.interaction.DragRotate`**, que permite hacer zoom y rotar la vista a la vez. Por defecto se activa al pulsar el tecla Shift y dependiendo de la dirección arrastramos el mapa de la interacción se aplica un zoom o de una rotación de el mapa.

- `ol.interaction.DragAndDrop`**.** El drag & drop permite arrastrar y soltar un archivo vectorial en el mapa y automáticamente carga su contenido como objeto espacial.

Podemos ampliar las interacciones de la siguiente forma:

```
var map = new ol.Map({
    ...
    interactions: ol.interaction.defaults().extend( [
                      new ol.interaction.DragBox()
                      ]),
    ...
});
```

mappingGIS

Otras interacciones

`ol.interaction.Select`

Interacción de selección → permite seleccionar datos vectoriales (*features)* dentro del mapa. Cada *feature* seleccionado se renderiza usando un estilo diferente, especificado por la propiedad **style**, y la interacción permite recuperar el conjunto de elementos seleccionados a través del método **getFeatures().**

Propiedades de la clase ol.interaction.Select

- `style` → los *features* seleccionados se renderizarán utilizando este estilo. Puede ser una instancia ol.Style o una función de estilos.
- `layers` → podemos restringir la capa de la que se deben seleccionar los *features*.
- `condition` → una función que determina si el evento mapa debe ser manejado por las interacciones. Por defecto es `ol.events.condition.singleClick`, lo que significa que al hacer clic en un *feature* se selecciona y se desactivan selecciones previas.
- `toggleCondition` → esta es la condición que deben cumplir para modificar el estado de un *feature*. Por defecto se ejecuta al pulsar la tecla *Shift*.
- `addCondition` y `removeCondition` → Permite especificar diferentes condiciones.

mappingGIS

Otras interacciones

ol.interaction.Draw

Para **dibujar nuevos *features*** en una capa vectorial, utilizamos la clase ol.interaction.Draw

Permite dibujar objetos geográficos utilizando un único tipo de geometría, especificado en propiedad obligatoria **Type**, y guardarlas en una fuente vectorial, especificado en propiedad **source**, que será una instancia ol.source.Vector donde los *features* se guardarán.

Ejemplo:

```
var drawPolygons = new ol.interaction.Draw({
    type: 'Polygon',
    source: polygonSource
});
```



Ejemplo de creación una herramienta de medición con la clase ol.interaction.Draw

mappingGIS

Otras interacciones

ol.interaction.Modify → Permite modificar objetos espaciales.

Requiere especificar un conjunto de características para trabajar con la propiedad obligatoria **feature.**

La modificación de objetos espaciales funciona mediante el uso de la clase `ol.interaction.Select` en combinación con la clase `ol.interaction.Modify`. Comparten una colección común (`ol.Collection`) de elementos. Los *features* seleccionados con `ol.interaction.Select` se convierten en candidatos para ser modificados con `ol.interaction.Modify`.

Ejemplo:

```
var select = new ol.interaction.Select({style: style});

var modify = new ol.interaction.Modify({
        features: select.getFeatures()
});
```



Ahora puedes hacer los ejercicios de esta unidad

mappingGIS